



**LIMBUS SPLIT PRO**  
WINDOWS EDITION

# Prompt maestro para Codex

Diseño, separación, reproducción y GPU,  
licencias, pruebas e instalador de Windows.

VERSIÓN 1.0 · JULIO 2026

# LIMBUS SPLIT PRO

## Prompt maestro para crear la edición nativa de Windows con Codex

Versión 1.0 - Julio de 2026

### Cómo utilizar este documento

1. Abre Codex en un ordenador Windows dentro de una carpeta de proyecto vacía.
2. Copia el prompt completo incluido a continuación.
3. Adjunta referencias visuales y archivos de audio de prueba si están disponibles.
4. Permite que Codex investigue, implemente, compile, pruebe y genere el instalador.
5. No distribuyas el resultado hasta verificar modelos, dependencias, firma y funcionamiento en una instalación limpia.

**IMPORTANTE:** Este documento contiene instrucciones de desarrollo. No incluye modelos entrenados ni concede derechos sobre software, datasets o pesos de terceros. Cada usuario es responsable de comprobar las licencias de todos los componentes que incorpore.

# PROMPT COMPLETO - COPIAR DESDE AQUÍ

Actúa como arquitecto de software, desarrollador senior de Windows, especialista en audio digital, machine learning aplicado a separación musical, diseño de producto, seguridad y empaquetado.

Quiero que diseñes, programes, pruebes y empaquetes desde cero una aplicación de escritorio para Windows llamada Limbus Split Pro.

Trabaja de forma autónoma hasta obtener una aplicación funcional y verificable. No te limites a describir código: inspecciona el entorno, investiga, implementa, compila, ejecuta pruebas reales y genera los artefactos finales.

Si encuentras una imposibilidad técnica o legal, no la ocultes ni simules que una función existe. Continúa con todo lo que sí pueda completarse y explica exactamente qué modelo, licencia, certificado, dispositivo o recurso falta.

## 1. Objetivo general

Limbus Split Pro será una aplicación nativa de Windows para separar una mezcla musical en pistas independientes mediante modelos locales de inteligencia artificial.

Debe estar dirigida a músicos, productores, alumnos y usuarios sin conocimientos técnicos. El flujo principal será: cargar una canción, elegir qué separar, escoger una carpeta, procesar, escuchar las pistas y exportar.

Prioridades, en este orden:

1. Calidad real de separación.
2. Sincronización exacta de reproducción y exportación.
3. Estabilidad y ausencia de archivos silenciosos falsos.
4. Facilidad de uso.
5. Privacidad y procesamiento local.
6. Compatibilidad con Windows 10 y Windows 11.
7. Portabilidad del instalador.
8. Licencias y atribuciones correctas.
9. Diseño minimalista, bonito y trabajado.

No construyas una página web, una PWA ni una aplicación Electron.

Desarrolla una aplicación nativa de Windows. Prefiere C# con una versión LTS vigente de .NET, WPF y patrón MVVM por su madurez, estabilidad y acceso directo al sistema de audio. Puedes utilizar WinUI 3 solamente si verificas que el empaquetado, la reproducción multipista y la compatibilidad con las versiones objetivo son igual de fiables.

Antes de fijar versiones, comprueba la versión LTS actual de .NET y las versiones compatibles de Windows App SDK, WPF, NAudio y las demás dependencias. Registra todas las versiones de forma reproducible.

## 2. Entorno de construcción

La compilación final y las pruebas de Windows deben ejecutarse en Windows real o en un runner oficial de Windows.

Si Codex está trabajando desde macOS o Linux:

- Puede preparar arquitectura, código y documentación.
- Debe crear una automatización o CI para compilar en Windows.
- No debe afirmar que el EXE o instalador funciona si no se ha probado en Windows.
- Debe entregar una lista exacta de las pruebas pendientes en un equipo Windows.

Configura un proyecto reproducible mediante solución y proyectos .NET, archivos de bloqueo de paquetes, scripts PowerShell y una estructura de carpetas clara.

## 3. Nombre, identidad y diseño

Nombre del producto: Limbus Split Pro.

El diseño debe ser:

- Minimalista.
- Limpio.
- Elegante.
- Profesional.
- Amigable.
- Con personalidad visual.
- Con una paleta de color reconocible y algo divertida, sin resultar infantil.

Evita una interfaz genérica, excesivamente gris o con aspecto de prototipo.

No utilices el eslogan “Una mezcla. Solo lo que necesitas”.

Utiliza Fluent Design como referencia visual sin convertir la aplicación en una copia genérica de Windows Settings.

Respetar:

- Barra de título y botones minimizar, maximizar y cerrar.
- Escalado de pantalla al 100 %, 125 %, 150 %, 175 % y 200 %.
- Temas claro y oscuro.
- Contraste alto.
- Navegación mediante teclado.
- Lectores de pantalla y nombres accesibles.
- Ventanas pequeñas y monitores 4K.

El logotipo, los títulos y los controles nunca deben solaparse con la barra de título.

## 4. Flujo principal

El usuario debe poder:

1. Arrastrar una canción o seleccionarla con un diálogo nativo.
2. Ver nombre, formato, duración, frecuencia de muestreo y canales.

3. Elegir exactamente qué instrumentos desea separar.
4. Elegir una carpeta de trabajo y exportación mediante FolderPicker.
5. Iniciar la separación.
6. Ver progreso real y etapa actual.
7. Cancelar de forma segura.
8. Escuchar simultáneamente todas las pistas generadas.
9. Aplicar mute, solo y volumen independiente.
10. Cambiar el punto de reproducción mediante una línea temporal.
11. Exportar pistas individuales.
12. Exportar una mezcla con los volúmenes, mute y solo actuales.
13. Abrir la carpeta de resultados en el Explorador de archivos.

Nunca guardes resultados en una ubicación desconocida. El usuario debe elegir explícitamente la carpeta antes de separar.

Admite rutas con espacios, caracteres Unicode, nombres largos y unidades distintas a C:. No construyas comandos concatenando cadenas sin escapar.

## 5. Selector de pistas

La interfaz debe disponer de controles “Todas” y “Ninguna”, además de selectores individuales.

Categorías deseadas:

- Voces.
- Voz principal.
- Coros y segundas voces.
- Efectos vocales, reverberación o ambiente vocal.
- Ruido o artefactos.
- Batería completa.
- Bombo.
- Caja.
- Toms.
- Platos.
- Bajo.
- Guitarra.
- Guitarra acústica, solo si existe un modelo realmente capaz.
- Guitarra eléctrica, solo si existe un modelo realmente capaz.
- Piano y teclados.
- Otros instrumentos que puedan separarse mediante modelos reales y verificables.
- Other.

No muestres opciones falsas ni generes archivos silenciosos para aparentar que una separación existe.

Si no existe un modelo técnicamente válido y con licencia adecuada:

- Deja la opción desactivada.
- Explica por qué no está disponible.
- No generes una pista vacía.
- Registra la limitación en el informe.

## 6. Lógica de Other

La lógica de complemento es obligatoria.

- Si el usuario selecciona solamente “Voces”, exporta “Voces.wav” y “Other.wav”.
- Other debe contener todo lo que no sea voz.
- Si selecciona solamente “Batería”, exporta “Batería completa.wav”, sus detalles cuando corresponda, y “Other.wav”.
- Other debe contener todo lo que no sea batería.
- Si selecciona varios instrumentos, Other debe contener todo aquello que no haya sido seleccionado.
- Si selecciona todas las categorías, genera Other solamente si queda material residual real.
- Other no debe ser un archivo silencioso creado artificialmente.
- La suma de las pistas principales y Other debe reconstruir la mezcla con el menor error posible.
- Evita duplicar material cuando se exporta simultáneamente una pista general y sus subpistas.

Para voces detalladas:

- Voz principal y coros deben proceder de una separación real.
- Los efectos vocales no deben ser una copia de la voz.
- El ruido debe ser una pista independiente real.
- Ejecuta internamente las etapas necesarias aunque el usuario no haya seleccionado sus resultados intermedios.
- No obligues al usuario a entender dependencias entre modelos.

## 7. Investigación y motor de separación

Investiga modelos actuales antes de escogerlos. Utiliza fuentes primarias: repositorios oficiales, model cards, publicaciones de los autores y licencias incluidas con los pesos.

Condiciones obligatorias:

- Procesamiento completamente local.
- Sin subir canciones.
- Sin llamadas a APIs de pago durante la separación.
- Sin descargar modelos silenciosamente mientras se procesa una pista.
- Gestión explícita de la instalación de modelos.
- Hash SHA-256 de cada modelo.

- Registro de procedencia, versión, licencia, autor y capacidades.
- Diferenciación entre licencia del código y licencia de los pesos.
- Prohibición de asumir que los pesos son comerciales porque el repositorio sea MIT.

En una compilación distribuable utiliza únicamente modelos cuyos pesos tengan autorización inequívoca para uso, inclusión en aplicaciones, redistribución y uso comercial cuando corresponda.

Implementa un manifiesto con:

- Identificador.
- Ruta relativa.
- Hash SHA-256.
- Tamaño esperado.
- Versión.
- Origen.
- Licencia.
- Evidencia de licencia.
- Autor y atribución.
- Capacidades.
- Autorización de redistribución.
- Autorización comercial.

Implementa un verificador fail-closed. La compilación pública debe fallar ante un modelo no registrado, alterado, sin procedencia, sin autorización clara o sin las capacidades requeridas.

Puede existir una compilación experimental marcada como:

“COMPILACIÓN LOCAL DE DESARROLLO - NO DISTRIBUIR”.

No reduzcas la calidad únicamente para resolver una licencia. Si no existe un sustituto equivalente, informa del bloqueo y deja preparada la arquitectura para conectar un modelo autorizado posteriormente.

## 8. Arquitectura del motor Python

La interfaz nativa de Windows y el motor de machine learning deben estar separados.

Arquitectura recomendada:

- Aplicación C# como proceso principal.
- Motor Python local como proceso secundario controlado.
- Comunicación mediante JSON Lines, pipes con nombre o un protocolo local documentado.
- Salida estándar reservada para eventos estructurados.
- Salida de error y logs técnicos separados.
- Cancelación mediante señal controlada y limpieza de procesos hijos.
- Nunca utilizar cmd.exe con comandos contruidos concatenando texto del usuario.
- Utilizar ProcessStartInfo.ArgumentList o un mecanismo equivalente seguro.

El cierre de la aplicación debe finalizar correctamente los procesos del motor sin dejar python.exe o procesos de inferencia ejecutándose.

## 9. Python propio y reproducible

No dependas del Python instalado por el usuario, Microsoft Store, Chocolatey, Conda, pyenv-win ni Visual Studio.

Incorpora un Python propio para Windows.

Requisitos:

- Versión exacta fijada.
- Fuente o distribución oficial verificable.
- URL y SHA-256.
- Arquitectura x64 como objetivo principal.
- Edición ARM64 separada solamente si todas las dependencias son ARM64 nativas.
- Ausencia de rutas del ordenador de compilación.
- PYTHONHOME y PYTHONNOUSERSITE configurados.
- Sin acceso a site-packages del usuario.
- Sin pip, setuptools o compiladores en el producto final, salvo necesidad demostrada.
- Cachés Python, Torch, ONNX y JIT fuera de Program Files.
- Registro de procedencia en PROVENANCE.json.

No copies una instalación arbitraria desde el ordenador del desarrollador. Construye o ensambla el runtime mediante un script reproducible y verifica todos sus binarios.

## 10. CPU, GPU y aceleración

La aplicación debe funcionar siempre en CPU, aunque sea más lenta.

Detecta de forma segura:

- CPU y memoria disponible.
- GPU NVIDIA compatible.
- CUDA/cuDNN solamente si la distribución elegida lo requiere y sus licencias permiten empaquetarlos.
- DirectML, ONNX Runtime u otra aceleración solamente cuando el modelo y la calidad estén verificados.

No presentes una opción de GPU que falle silenciosamente.

Requisitos:

- Selector Automático, CPU y GPU cuando corresponda.
- Fallback automático a CPU con explicación clara.
- Prueba previa de memoria antes de procesar canciones largas.
- División en segmentos sin perder continuidad.
- Mensajes comprensibles ante memoria insuficiente.

- Misma frecuencia de muestreo, duración y alineación independientemente del dispositivo.
- Comparación numérica entre CPU y GPU para detectar degradaciones importantes.

Evita incluir varios gigabytes de runtimes GPU si pueden suministrarse como paquete opcional. La edición CPU debe ser plenamente funcional.

## 11. Dependencias y bibliotecas nativas

Reduce dependencias sin afectar calidad o estabilidad.

Elimina de la distribución:

- pip.
- setuptools.
- Cython.
- Tests.
- Ejemplos.
- Compiladores.
- Cabeceras de desarrollo innecesarias.
- Backends no utilizados.
- DLL de arquitecturas incorrectas.
- Archivos PDB de terceros no necesarios para el usuario final.

No elimines una dependencia sin ejecutar después pruebas reales.

Audita todas las DLL y ejecutables:

- Arquitectura x64 o ARM64 correcta.
- Dependencias de Visual C++ Runtime.
- Rutas de búsqueda seguras.
- Ausencia de DLL faltantes.
- Ausencia de dependencias cargadas desde carpetas del usuario o del PATH global.
- Hashes reproducibles.
- Licencias compatibles.

Evita vulnerabilidades de DLL search-order. Carga bibliotecas únicamente desde ubicaciones controladas y utiliza APIs seguras de búsqueda de DLL.

## 12. FFmpeg, libsndfile y formatos

Admite al menos WAV de forma nativa y fiable.

Si incorporas FFmpeg u otras bibliotecas para MP3, FLAC, AAC o M4A:

- Determina si la compilación es LGPL o GPL.
- No incluyas codecs GPL en una aplicación cerrada sin evaluar las consecuencias.
- Incluye textos de licencia y fuentes cuando corresponda.

- Registra opciones de compilación, versión y hash.
- Permite sustituir bibliotecas LGPL cuando sea obligatorio.

No confíes en que el usuario tenga FFmpeg instalado en PATH.

## 13. LGPL, atribuciones y SBOM

Si una biblioteca LGPL es necesaria:

- Utiliza enlace dinámico cuando corresponda.
- Incluye la licencia completa.
- Incluye el código fuente exacto o una oferta válida.
- Proporciona instrucciones de sustitución.
- Documenta cómo reconstruir y volver a firmar.
- Registra hash y versión.

Genera automáticamente:

- THIRD\_PARTY\_NOTICES.txt.
- SBOM CycloneDX JSON.
- Carpeta con licencias completas.
- Manifiesto de modelos.
- Informe de cumplimiento.
- Instrucciones LGPL.

Genera el SBOM desde el contenido real y saneado de la versión final, no desde una lista manual.

## 14. Reproducción de audio en Windows

La reproducción debe comportarse como un mezclador multipista sencillo y preciso.

Utiliza un solo dispositivo de salida, un solo reloj y una sola línea temporal. Una opción adecuada es NAudio con WASAPI y un único MixingSampleProvider, siempre que la sincronización se demuestre mediante pruebas.

No abras un reproductor independiente y sin coordinación para cada pista.

Todas las pistas deben:

- Comenzar en el mismo sample.
- Conservar duración, canales y frecuencia de muestreo compatibles.
- Mantenerse sincronizadas durante toda la reproducción.
- Responder juntas a play, pausa, stop y seek.
- Compartir el mismo contador de posición.

Permite seleccionar el dispositivo de audio de Windows y reaccionar correctamente si se desconecta o cambia el dispositivo predeterminado.

Usa WASAPI compartido como modo estable por defecto. Añade modo exclusivo únicamente si está realmente probado y aporta valor.

## 15. Controles y comportamiento en tiempo real

Controles generales:

- Play.
- Pausa.
- Stop con semántica clara.
- Barra de progreso.
- Tiempo actual.
- Duración total.
- Clic y arrastre para hacer seek.
- Tecla espacio para alternar play y pausa.

El atajo de espacio no debe activarse al escribir en campos de texto ni mientras un diálogo modal está abierto.

Comportamiento obligatorio:

- Mute no detiene la reproducción.
- Solo no detiene la reproducción.
- Volumen no detiene la reproducción.
- Estos cambios no devuelven el cursor al inicio.
- Seek continúa desde el punto seleccionado.
- Seek no produce rebote, doble ataque, clic o silencio perceptible.
- Utiliza crossfade o rampas de ganancia cortas.
- No reconstruyas innecesariamente todo el dispositivo de audio.
- Play después de pausa continúa desde la posición actual.

Gestiona cambios de dispositivo, suspensión del sistema y retorno desde suspensión sin dejar el motor bloqueado.

## 16. Mezclador

Cada pista debe mostrar:

- Nombre.
- Icono o color.
- Botón Mute.
- Botón Solo.
- Control de volumen.
- Porcentaje o valor de volumen.
- Medidor opcional, solamente si funciona con fluidez.

Permite uno o varios solos.

Cuando exista al menos un solo activo, solamente deben sonar las pistas en solo, excepto las que también estén en mute.

Los cambios deben utilizar rampas para evitar clics.

## 17. Exportación de mezcla

Añade un botón “Exportar mezcla”.

La mezcla exportada debe respetar exactamente:

- Volumen por pista.
- Mute.
- Solo.
- Alineación.
- Duración.
- Frecuencia de muestreo.
- Canales.
- Punto inicial común.

Realiza la exportación offline con precisión de samples. No grabes la salida del dispositivo en tiempo real.

Evita clipping:

- Calcula el pico final.
- Avisa o aplica protección solamente cuando sea necesario.
- No normalices silenciosamente.
- Documenta cualquier limitador o reducción de ganancia.

Exporta como mínimo WAV PCM.

## 18. Interfaz

Panel izquierdo:

1. Archivo de entrada.
2. Qué extraer.
3. Carpeta de trabajo.
4. Dispositivo CPU/GPU cuando corresponda.
5. Botón “Separar y exportar”.

Panel derecho:

1. Estado del proceso.
2. Reproductor general.
3. Barra temporal.
4. Lista de pistas.
5. Mute, Solo y Volumen.
6. Botón “Exportar mezcla”.

### 7. Botón “Abrir carpeta”.

Durante la separación muestra etapa, progreso, modelo, dispositivo y errores comprensibles.

No muestres una traza Python como único mensaje. Guarda el detalle en logs y presenta una explicación clara con una posible solución.

## 19. Rutas, permisos y datos mutables

Una aplicación instalada debe considerarse de solo lectura.

Nunca escribas modelos, cachés o temporales dentro de:

C:\Program Files\Limbus Split Pro

Utiliza ubicaciones de Windows apropiadas:

- %LOCALAPPDATA%\Limbus Split Pro para datos por usuario.
- %LOCALAPPDATA%\Limbus Split Pro\Models para modelos descargables por usuario.
- %LOCALAPPDATA%\Limbus Split Pro\Cache para cachés.
- %TEMP% para temporales de proceso.
- %PROGRAMDATA% solamente para datos compartidos que realmente lo necesiten.
- Carpeta elegida por el usuario para exportaciones.

No requieras privilegios de administrador para procesar audio.

Los temporales deben tener nombres únicos, limpiarse tras éxito y conservarse de manera controlada ante un fallo útil para diagnóstico.

Gestiona correctamente OneDrive, carpetas de red y unidades externas. No asumas que todos los archivos son locales o que permanecen disponibles durante todo el proceso.

## 20. Instalador de Windows

Genera:

- Aplicación portable de desarrollo, si resulta útil.
- Instalador MSI o Setup EXE profesional.
- Checksums SHA-256.
- Informe de contenido instalado.

Evalúa WiX Toolset para MSI y un bootstrapper solamente si es necesario. MSIX puede utilizarse si sus restricciones de archivos, modelos grandes, actualizaciones y datos mutables encajan con la arquitectura.

El instalador debe:

- Mostrar editor, versión y destino.
- Instalar en Program Files para instalación por máquina o en LocalAppData para instalación por usuario.
- No sobrescribir documentos del usuario.
- Crear accesos directos correctos.
- Registrar una desinstalación limpia.

- Conservar proyectos y exportaciones al desinstalar.
- Eliminar solamente cachés si el usuario lo solicita.
- Actualizar sin duplicar instalaciones.
- Verificar espacio disponible.
- Evitar descargas ocultas durante la instalación.

Prueba instalación, actualización, reparación y desinstalación.

## 21. Firma, SmartScreen y seguridad

Añade soporte para firma Authenticode de:

- Ejecutable principal.
- DLL propias.
- Procesos auxiliares.
- Instalador MSI o Setup EXE.

Utiliza un certificado de firma de código válido y timestamp RFC 3161.

No presentes una firma autofirmada como distribución pública final.

Verifica la firma mediante SignTool y PowerShell. Comprueba el comportamiento de Microsoft Defender SmartScreen en una máquina limpia.

No solicites exclusiones de antivirus como solución normal. Si Defender detecta un falso positivo:

- Reduce empaquetadores opacos.
- Evita ejecutables autoextraíbles sospechosos.
- Firma correctamente.
- Envía el archivo a revisión de Microsoft.
- Documenta el hash afectado.

No almacenes secretos de firma ni contraseñas en el repositorio.

## 22. Logs, errores y recuperación

Guarda logs rotativos en una carpeta por usuario.

No registres canciones, credenciales, rutas privadas completas o contenido sensible sin necesidad.

Los mensajes deben diferenciar:

- Modelo ausente.
- Hash incorrecto.
- Formato no compatible.
- Memoria insuficiente.
- GPU no compatible.
- Permiso denegado.

- Archivo bloqueado por otra aplicación.
- Ruta de red desconectada.
- Dispositivo de audio no disponible.
- Cancelación del usuario.

La aplicación debe poder recuperarse de un fallo sin exigir reiniciar Windows.

## 23. Pruebas obligatorias

No consideres terminada la aplicación porque compile.

Realiza pruebas en Windows 10 y Windows 11, al menos en x64.

Comprueba:

1. Compilación reproducible desde un checkout limpio.
2. Arranque sin Visual Studio instalado.
3. Arranque sin Python del sistema.
4. Python, SSL y Izma integrados.
5. PyTorch u ONNX Runtime.
6. Motor principal de separación.
7. Modelos vocales.
8. Separación de ruido.
9. Separación de efectos.
10. Separación detallada de batería.
11. CPU.
12. GPU compatible, si se ofrece.
13. Fallback de GPU a CPU.
14. Duración de todas las pistas.
15. Frecuencia de muestreo.
16. Canales.
17. Archivos no silenciosos cuando exista señal.
18. Sincronización sample a sample.
19. Reconstrucción de la mezcla.
20. Mute durante reproducción.
21. Solo durante reproducción.
22. Volumen durante reproducción.
23. Seek suave.
24. Barra temporal.
25. Atajo de espacio.
26. Exportación de mezcla.

27. Cambio de dispositivo de audio.
28. Suspensión y reanudación de Windows.
29. Rutas Unicode y largas.
30. Carpetas de OneDrive.
31. Archivos bloqueados.
32. Cancelación durante cada etapa.
33. Instalación limpia.
34. Actualización desde una versión anterior.
35. Desinstalación.
36. Ejecución como usuario sin privilegios.
37. Ausencia de escritura en Program Files.
38. Ausencia de rutas del ordenador de compilación.
39. Arquitectura correcta de EXE y DLL.
40. Dependencias DLL completas.
41. Firma Authenticode.
42. SmartScreen y Microsoft Defender.
43. SBOM y licencias.
44. Hashes de modelos.
45. Ausencia de modelos no autorizados en la compilación pública.

No consideres un archivo silencioso como prueba correcta.

Utiliza grabaciones que contengan realmente cada instrumento. Para batería detallada, utiliza una grabación con bombo, caja, toms y platos.

## 24. Automatización y CI

Crea una pipeline de Windows que:

- Restaure dependencias desde versiones bloqueadas.
- Compile en Release x64.
- Ejecute tests unitarios e integración.
- Verifique arquitectura de binarios.
- Analice dependencias DLL.
- Genere SBOM.
- Compruebe licencias y modelos.
- Construya el instalador.
- Firme únicamente en un entorno protegido cuando existan certificados.
- Publique hashes y artefactos.

La pipeline pública nunca debe incluir secretos ni modelos cuya redistribución no esté autorizada.

## 25. Informes

Después de cada revisión importante entrega un informe con:

- Funciones implementadas.
- Modelos utilizados.
- Licencias verificadas.
- Dependencias eliminadas.
- Pruebas realizadas.
- Equipos y versiones de Windows probados.
- Resultados CPU y GPU.
- Errores encontrados y corregidos.
- Limitaciones pendientes.
- Estado comercial.
- Estado de firma.
- Estado de SmartScreen.
- Hashes de instaladores.

## 26. Entrega en cada iteración

Después de cada petición proporciona enlaces a:

- Carpeta o ZIP de desarrollo.
- Ejecutable portable, si existe.
- MSI o Setup EXE.
- Informe de pruebas.
- SBOM y avisos legales.

No presentes como “lista para vender” una compilación que:

- Utilice pesos con licencia ambigua.
- No esté firmada.
- No haya superado una instalación limpia.
- Escriba dentro de Program Files.
- Dependenda del ordenador del desarrollador.
- Sea detectada por Defender sin haber investigado el motivo.

## 27. Forma de trabajo

Antes de cambiar archivos:

- Inspecciona el estado actual.
- Conserva modificaciones existentes.

- Evita operaciones destructivas.
- Documenta decisiones técnicas.

Verifica versiones, licencias y requisitos de Windows mediante fuentes oficiales actuales.

Toma decisiones razonables para avanzar, pero no sustituyas una función solicitada por otra más simple solamente para conseguir que las pruebas pasen.

## 28. Criterio final de éxito

Limbos Split Pro para Windows estará terminado cuando:

- Sea una aplicación nativa y visualmente cuidada.
- Separe únicamente pistas que realmente pueda generar.
- Las pistas contengan audio real.
- Other funcione como complemento correcto.
- El mezclador sea sample-accurate.
- Mute, solo, volumen y seek funcionen sin detener la reproducción.
- La mezcla exportada coincida con lo escuchado.
- Funcione en Windows 10 y Windows 11 sin herramientas de desarrollo.
- No dependa de Python, Chocolatey o Visual Studio del usuario.
- Incorpore un runtime reproducible.
- Funcione en CPU y gestione correctamente la aceleración opcional.
- No escriba en Program Files.
- Incluya atribuciones, SBOM y cumplimiento LGPL.
- Distribuya únicamente modelos con licencia inequívoca.
- Tenga instalador, actualización y desinstalación verificadas.
- Esté firmada con Authenticode antes de considerarla comercial.

## 29. Primera ejecución

Empieza en este orden:

1. Inspecciona el workspace y el sistema operativo actual.
2. Crea un inventario técnico.
3. Investiga modelos y prepara una tabla de licencias.
4. Elige WPF o WinUI 3 justificando la decisión.
5. Diseña la arquitectura C# y Python.
6. Presenta un plan de implementación.
7. Construye la primera versión funcional.
8. Ejecuta pruebas reales de audio.
9. Corrige los fallos.

10. Genera el instalador de Windows.
11. Ejecuta una instalación limpia.
12. Entrega artefactos e informe verificable.

Continúa trabajando hasta entregar el mejor resultado posible, sin simular capacidades inexistentes ni ocultar limitaciones legales, técnicas o de compatibilidad.

## FIN DEL PROMPT

### Aviso de distribución

Este prompt puede publicarse gratuitamente para que cada usuario lo utilice con su propia cuenta de Codex.

No compartas claves API, credenciales, certificados de firma ni cuentas. El código, las dependencias y los modelos incorporados durante la construcción pueden estar sujetos a licencias de terceros y deben revisarse antes de distribuir la aplicación resultante.

Codex es un producto de OpenAI. Este documento es independiente y no implica afiliación, patrocinio ni respaldo oficial de OpenAI.